
KNOWLEDGE GRAPH FOUNDATION MODELS FOR RELATIONAL DATABASE LEARNING

A PREPRINT

Marawan Eldeib

Institute of Artificial Intelligence
University of Stuttgart
Stuttgart, Germany
st196189@stud.uni-stuttgart.de

Rongchuan Zhang

Institute of Artificial Intelligence
University of Stuttgart
Stuttgart, Germany
st191486@stud.uni-stuttgart.de

Sanan Akther

Institute of Artificial Intelligence
University of Stuttgart
Stuttgart, Germany
st191505@stud.uni-stuttgart.de

February 4, 2026

ABSTRACT

Knowledge Graph Foundation Models (KGFM) enable zero-shot reasoning over graphs with unseen entities and relations, but their effectiveness on relational database tasks remains unexplored. This work presents the first systematic comparison of random-walk-based (Flock) and message-passing-based (ULTRA/POSTRA) KGFM on link prediction, classification, and regression tasks derived from RelBench F1. For link prediction, Flock achieves 0.894 MRR, outperforming ULTRA by 17.6% in zero-shot transfer—with Flock’s zero-shot performance exceeding ULTRA’s fine-tuned results. For regression, both paradigms successfully predict continuous values when formulated as link prediction over discretized ranges. Classification evaluation was limited by methodological challenges in the relational-to-KG conversion, revealing important requirements for temporal edge splitting and candidate set design in future work. Our results demonstrate that KGFM, particularly random-walk-based approaches, transfer effectively to relational database link prediction, suggesting potential for reducing task-specific model development in this domain.

1 Introduction

Foundation models have transformed language and vision by enabling zero-shot generalization through transferable representations. Knowledge Graph Foundation Models (KGFM) aim to bring similar capabilities to knowledge graphs, enabling inference over graphs with arbitrary entity and relation vocabularies [1, 2]. Unlike traditional transductive knowledge graph embedding methods, KGFM seek to generalize to unseen entities and relations by learning structural invariants [3]. Recent work formalizes this inductive ability as *double-equivariance*, requiring invariance under permutations of both entities and relations [2].

Two contrasting paradigms have emerged for KGFM. ULTRA constructs relation representations as functions of their interactions by building a graph-of-relations and applying graph neural networks with labeling strategies [1]. By avoiding fixed entity and relation embeddings, a single pretrained ULTRA model can generalize to unseen knowledge graphs with arbitrary schemas. When combined with an NBFNet-based predictor, ULTRA achieves strong zero-shot link prediction performance, with additional gains from fine-tuning [4]. POSTRA extends this framework to temporal knowledge graphs by introducing positional time-aware representations, enabling inductive generalization to unseen entities, relations, and timestamps [5]. Both ULTRA and POSTRA follow a neighbor-aggregation paradigm based on message passing.

In contrast, Flock abandons message passing in favor of random-walk-based reasoning. It introduces probabilistic node–relation equivariance by sampling random walks conditioned on a query and encoding the resulting sequences using a sequence model [2]. Aggregating multiple walks yields query-specific representations without relying on fixed entity or relation embeddings. This stochastic formulation allows Flock to break structural symmetries while retaining strong inductive bias. Empirically, Flock is a universal approximator for link prediction functions and achieves state-of-the-art performance across a wide range of knowledge graph benchmarks [2].

While KGfMs have largely been evaluated on link prediction, there is growing interest in applying graph learning to relational databases. Relational Deep Learning (RDL) converts relational schemas into graphs and applies GNNs to tasks such as classification and regression [6, 7]. RelBench provides standardized relational datasets and tasks to support this line of research [8]. Initial studies demonstrate that GNN-based RDL models outperform expert-engineered features while substantially reducing human effort [9, 10]. The RelBench F1 dataset, for example, includes driver DNF classification, podium prediction, and position regression tasks [8].

This work bridges these areas by evaluating whether KGfMs can serve as foundation models for relational databases. We introduce a unified experimental pipeline that benchmarks Flock and ULTRA/POSTRA on link prediction, classification, and regression tasks derived from RelBench F1, providing the first systematic comparison between random-walk-based and message-passing-based KGfMs beyond link prediction.

Our experiments yield three findings. First, Flock achieves strong link prediction performance (0.894 MRR), outperforming ULTRA by 17.6% in zero-shot transfer. Second, both paradigms successfully handle regression when formulated as link prediction over discretized value ranges. Third, our classification experiments revealed methodological challenges in the relational-to-KG conversion process.

2 Related Work

2.1 Relational Databases and Graph Learning

The RELBENCH benchmark established a suite of tasks for evaluating machine learning on relational databases by representing them as graphs [8]. Each relational database is converted into a heterogeneous graph with entities as nodes and foreign key relations as edges. Relational Deep Learning (RDL) methods apply Graph Neural Networks (GNNs) directly to these graphs, reducing reliance on manual feature engineering [6]. Empirical studies show that end-to-end learned models can outperform expert-crafted pipelines while substantially reducing human effort [9]. However, most RDL models are trained from scratch on each database. Our work explores using pretrained KG foundation models in this setting.

2.2 Knowledge Graph Foundation Models

KGfMs address inductive reasoning across unseen graphs by learning transferable structural representations. Early inductive approaches, such as GraIL and NBFNet, focused on unseen entities but assumed a fixed relation vocabulary [11, 4]. ULTRA extended this paradigm by learning relation representations via a graph-of-relations, enabling inductive generalization to unseen schemas [1]. POSTRA further extends this framework to temporal knowledge graphs by incorporating time-aware positional encodings [5]. Our evaluation uses ULTRA and POSTRA as representative neighbor-aggregation-based KGfMs.

2.3 Flock – Random Walks and Probabilistic Equivariance

Flock introduces a random-walk-based alternative to message passing for KG reasoning. By sampling query-conditioned random walks and encoding them with a sequence model, Flock achieves probabilistic equivariance and avoids fixed entity or relation embeddings [2]. This stochastic formulation enables Flock to break structural symmetries while retaining strong generalization. Theoretical analysis shows that Flock is a universal approximator for isomorphism-invariant link prediction functions, and empirical results demonstrate strong performance across diverse knowledge graphs [2].

2.4 Temporal Knowledge Graphs

Temporal knowledge graphs extend static KGs by associating facts with timestamps. POSTRA bridges KG foundation models to temporal settings by incorporating positional time-aware representations and message passing conditioned on temporal context [5]. This enables inductive generalization to unseen timestamps, entities, and relations. While our

primary focus is on static relational tasks, POSTRA provides an important extension for handling temporal structure in real-world databases.

In summary, Flock and ULTRA/POSTRA represent two complementary paradigms for KG foundation models. Prior work established their effectiveness on link prediction benchmarks. Our work extends this evaluation to relational database tasks, comparing random-walk-based and message-passing-based KGfms in this setting and identifying key challenges in experimental design.

3 Task Formulation

We study *supervised temporal relational reasoning* on relational databases under the RelBench setting. Each task is formulated as a time-dependent query over entities and relations, and solved via graph-based transductive reasoning.

3.1 Relational Database Setting

A RelBench dataset is provided as a relational database consisting of multiple tables connected via *primary key–foreign key* (PK–FK) relationships, together with a collection of supervised downstream tasks. From the database, we derive a set of entities $\mathcal{E}$, relations $\mathcal{R}$ induced by table-level PK–FK links, and timestamps $\mathcal{T}$.

We convert each PK–FK relationship into a directed relational edge (h, r, t) , where $h, t \in \mathcal{E}$ and $r \in \mathcal{R}$. If a tuple is associated with a timestamp $\tau \in \mathcal{T}$, we extend the relation to a temporal quadruple:

$$(h, r, t, \tau). \quad (1)$$

Aggregating all such quadruples across tables yields a *global temporal graph* representing the entire database.

3.2 Task-as-Relation Reformulation

RelBench tasks are provided independently from the relational schema. Each task specifies a supervised prediction target associated with an entity and a timestamp. To enable graph-based transductive models, we introduce a *task-specific relation* r_{task} and reformulate each task as:

$$(h, r_{\text{task}}, ?, \tau), \quad (2)$$

where h is the subject entity, τ is the query timestamp, and the tail entity encodes the prediction target.

3.3 Classification Tasks

For classification tasks, the ground-truth label is discrete. For example, in `driver-top3`, each sample can be written as $(\text{driver_id}, \tau, y)$ with $y \in \{0, 1\}$. We define a task relation *isTop3* and reformulate the query as:

$$(\text{driver_id}, \text{isTop3}, ?, \tau). \quad (3)$$

The tail entities correspond to discrete outcome states (e.g., True/False). The correct outcome is treated as the positive tail entity, while all other candidates are negative.

3.4 Regression Tasks

For regression tasks, the target is continuous. For example, in `driver-position`, each sample is $(\text{driver_id}, \tau, y)$ with $y \in \mathbb{R}$. We define a task relation *hasPosition* and construct:

$$(\text{driver_id}, \text{hasPosition}, ?, \tau). \quad (4)$$

Here, tail entities correspond to discretized integer position values, while the ground-truth target remains continuous. Regression is performed by predicting a distribution over candidate tails and computing an expected value.

4 Methodology

Our methodology consists of three consecutive modules: (1) relational data processing and temporal graph construction, (2) a time-aware extension of Flock, and (3) a unified adaptation framework built upon Flock/POSTRA for supervised classification and regression.

4.1 Relational Data Processing and Task-aware Graph Construction

RelBench datasets are provided as relational databases consisting of multiple tables connected via primary key–foreign key (PK–FK) relationships. To enable graph-based transductive reasoning, we convert the relational database into a temporal graph representation.

Each PK–FK relationship is interpreted as a directed relation (h, r, t) between entities, where h and t correspond to the foreign key and primary key entities, respectively. If a tuple is associated with a timestamp τ , the relation is extended to a temporal quadruple:

$$(h, r, t, \tau). \quad (5)$$

All such quadruples across tables jointly define a global temporal graph.

RelBench tasks are not part of the original relational schema. To integrate task supervision into the graph, we introduce a task-specific relation r_{task} for each task and reformulate it as a query of the form:

$$(h, r_{\text{task}}, ?, \tau), \quad (6)$$

where h is the subject entity and τ denotes the query timestamp. The prediction target is represented as the tail entity, enabling unified score-based inference for both classification and regression tasks.

Transductive setting: We adopt a transductive learning setup where all entities are observed during training, but specific task edges are held out for evaluation. The global temporal graph should contain only edges available at training time; edges corresponding to validation and test queries must be excluded to prevent information leakage.

4.2 Time-aware Flock

Flock is originally designed for static relational reasoning. We extend it to temporal settings by explicitly encoding timestamps and injecting them into the quadruple representation.

Temporal indexing: We focus on *temporal order* rather than absolute time intervals. Each timestamp τ is assigned an integer ID according to chronological order:

$$\tau \mapsto \text{id}(\tau). \quad (7)$$

Time embedding: We use a positional encoder to obtain a dense time embedding:

$$\mathbf{e}_\tau = \text{PE}(\text{id}(\tau)). \quad (8)$$

Quadruple representation and scoring: Given a temporal quadruple (h, r, t, τ) , we combine head, relation, tail, and time embeddings:

$$\mathbf{q}_{h,r,t,\tau} = \mathbf{e}_h + \mathbf{e}_r + \mathbf{e}_t + \mathbf{e}_\tau, \quad (9)$$

and compute a plausibility score with a scoring function $f(\cdot)$:

$$s(h, r, t, \tau) = f(\mathbf{q}_{h,r,t,\tau}). \quad (10)$$

This time-aware Flock scorer is shared across all tasks.

4.3 Model-Agnostic Adaptation Framework for Classification and Regression

The proposed adaptation framework is *model-agnostic* and applies to both Flock and POSTRA. It only assumes that the underlying model produces a scalar score for a temporal quadruple (h, r, t, τ) , and therefore does not rely on any specific message passing or reasoning mechanism.

As a result, the same adaptation strategy can be directly applied to Flock-based scoring models as well as POSTRA-style path reasoning models.

Full candidate supervision: Given a task query $(h, r_{\text{task}}, ?, \tau)$, the model computes scores for all candidate tail entities:

$$s_t = s(h, r_{\text{task}}, t, \tau), \quad \forall t \in \mathcal{E}. \quad (11)$$

The correct tail entity is treated as the positive sample, while all remaining candidates are treated as negatives. No heuristic negative sampling is used.

Probability normalization: Candidate scores are normalized into a probability distribution using a softmax function:

$$p_t = \frac{\exp(s_t)}{\sum_{t'} \exp(s_{t'})}. \quad (12)$$

4.3.1 Binary Classification

For classification tasks, the model is trained using cross-entropy loss over candidate tail entities:

$$\mathcal{L}_{\text{cls}} = - \sum_t y_t \log p_t, \quad (13)$$

where $y_t \in \{0, 1\}$ denotes the ground-truth label for candidate t . At test time, performance is evaluated using AUROC, which measures the ranking quality between positive and negative outcomes.

4.3.2 Regression via Expected Value

For regression tasks, each candidate tail entity t is associated with a discretized integer value $\text{val}(t)$ (e.g., a position index). The predicted continuous output is computed as the expectation:

$$\hat{y} = \sum_t p_t \cdot \text{val}(t). \quad (14)$$

The training and evaluation metric is mean absolute error (MAE):

$$\mathcal{L}_{\text{reg}} = |\hat{y} - y|. \quad (15)$$

5 Experiments

We evaluate KGFM s on the RelBench F1 dataset, comparing Flock (random-walk-based) and ULTRA/POSTRA (message-passing-based) on link prediction and downstream classification/regression tasks.

5.1 Dataset

We evaluate on RelBench F1, a knowledge graph derived from a Formula 1 racing relational database [8]. This dataset captures relational structure among drivers, races, constructors, circuits, and results through foreign key relationships. Table 1 summarizes the link prediction splits.

Table 1: Link prediction data splits for RelBench F1.

Split	Triples
Train	271,242
Validation	33,735
Test	33,865
Total	338,842

For downstream evaluation, we consider three tasks as shown in Table 2: DNF prediction (whether a driver fails to finish), Top3 prediction (whether a driver achieves a podium finish), and Position regression (predicting exact finishing position).

Table 2: Downstream task statistics. DNF and Top3 are binary classification; Position is regression.

Task	Type	Train	Valid	Test	Target
DNF	Classification	11,411	566	702	Did not finish
Top3	Classification	1,353	588	726	Podium finish
Position	Regression	7,453	499	760	Finishing position

5.2 Evaluation Protocol

For link prediction, we use standard ranking metrics with filtered evaluation, where known true triples are removed from the candidate set. Mean Reciprocal Rank (MRR) computes the average of inverse ranks of correct entities. Mean Rank (MR) reports the average rank of correct entities, where lower values indicate better performance. Hits@ k measures the proportion of correct entities ranked within the top k positions.

For classification, we report Area Under ROC Curve (AUROC), which measures the probability that a randomly chosen positive example is ranked higher than a negative one. For regression, we report Mean Absolute Error (MAE), defined as $\frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$, measuring average prediction error in position units.

5.3 Implementation

All experiments are implemented in PyTorch with PyTorch Geometric. Training uses AdamW optimization with random seeds fixed at 1024 for reproducibility. Table 3 provides the hyperparameter specifications for both models.

Table 3: Hyperparameters for both models.

Parameter	Flock	ULTRA/POSTRA
Architecture	GRU	NBFNet / IndNBFNet
Number of layers/steps	6	6
Hidden dimension	64	64
Number of walks	256	–
Walk length	128	–
Message function	–	DistMult
Aggregation	–	sum
Multi-hop depth	–	3
Learning rate		5×10^{-4}
Batch size		16
Epochs		10

5.4 Results

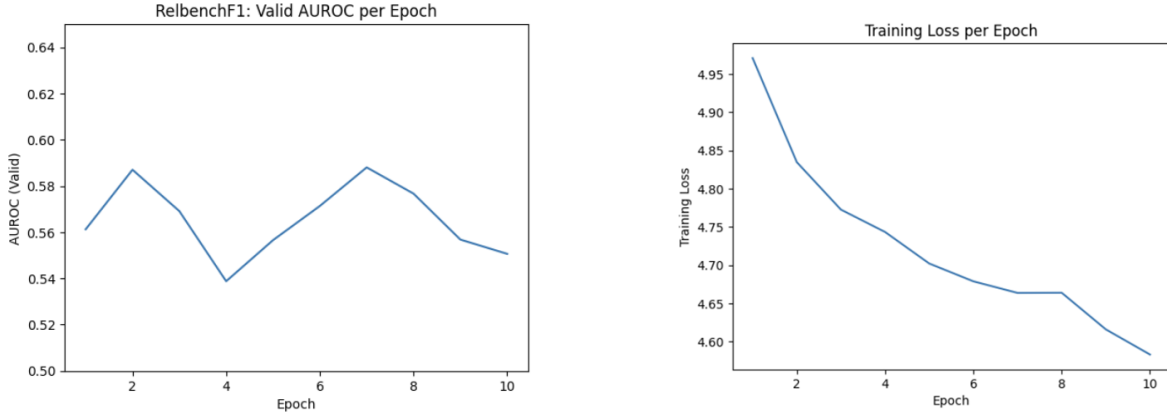
5.4.1 Link Prediction

Table 4 presents link prediction results. Flock outperforms ULTRA in both zero-shot and fine-tuned settings. In zero-shot evaluation, Flock achieves 0.885 MRR compared to ULTRA’s 0.752, representing a 17.6% relative improvement. After fine-tuning, Flock reaches 0.894 MRR with Hits@10 of 0.992.

Table 4: Link prediction results on RelBench F1. Best results in **bold**.

Model	Setting	MRR↑	MR↓	H@1↑	H@3↑	H@10↑
ULTRA	Zero-shot	0.752	470.71	0.725	0.761	0.805
ULTRA	Fine-tuned	0.878	4.94	0.843	0.897	0.947
Flock	Zero-shot	0.885	1.708	0.829	0.929	0.982
Flock	Fine-tuned	0.894	1.481	0.835	0.944	0.992

Two observations emerge from these results. First, Flock’s zero-shot performance (0.885 MRR) exceeds ULTRA’s fine-tuned result (0.878 MRR), suggesting that random-walk-based representations transfer more effectively to this relational domain. Second, fine-tuning yields modest gains for Flock (+1.0%) but substantial improvement for ULTRA (+16.7%), indicating that Flock’s pretrained weights already capture structural patterns relevant to relational database-derived graphs.



(a) Classification (Top3): Validation AUROC over epochs.

(b) Regression (Position): Training MAE over epochs.

Figure 1: Training dynamics across tasks. Classification shows oscillating AUROC without improvement, while regression shows consistent MAE reduction.

5.4.2 Classification

Table 5 presents classification results. Both models achieve performance near random chance (0.50 AUROC). On DNF prediction, POSTRA reaches 0.511 AUROC after fine-tuning, while Flock-time achieves 0.498 AUROC. Similar results are observed for Top3 prediction, where POSTRA achieves 0.539 AUROC and Flock-time achieves 0.487 AUROC.

Table 5: Classification results (AUROC $\uparrow$). Both models perform near random chance.

Task	Model	Zero-shot	Fine-tuned
DNF	POSTRA	0.484	0.511
	Flock-time	0.504	0.498
Top3	POSTRA	0.484	0.539
	Flock-time	0.511	0.487

These results contrast with RelBench baselines, which achieve 68.6% AUROC on DNF and 73.9% AUROC on Top3 using GNN-based approaches [8]. The source of this performance gap is examined in Section 5.5.

5.4.3 Regression

Table 6 presents regression results. Flock-time achieves MAE of 4.554 after fine-tuning, while POSTRA achieves 6.720. Both results compare favorably to the RelBench baseline of 11.926 [8].

Table 6: Position regression results (MAE $\downarrow$).

Model	Zero-shot	Fine-tuned
POSTRA	8.393	6.720
Flock-time	8.823	4.554

The regression task uses expected-value computation over candidate positions, which aligns with the soft ranking that link prediction models naturally produce.

5.4.4 Training Dynamics

Figure 1 illustrates learning behavior across tasks. Classification AUROC oscillates between 0.54 and 0.59 without consistent improvement, remaining near random chance despite decreasing training loss. In contrast, regression MAE decreases steadily from 4.97 to 4.58, indicating successful learning. This divergence between classification and regression training dynamics provides insight into how KGfMs interact with different task formulations.

5.5 Analysis and Limitations

5.5.1 Link Prediction Performance

Flock consistently outperforms ULTRA on link prediction across both evaluation settings. Two factors may explain this difference. First, random walks capture multi-hop relational patterns without explicit message passing, which may better suit the sparse structure of relational database-derived graphs. Second, Flock’s pretrained representations appear to transfer more effectively to this domain, as evidenced by its zero-shot performance exceeding ULTRA’s fine-tuned result.

The modest fine-tuning improvement for Flock (+1.0%) compared to ULTRA (+16.7%) further supports this interpretation. ULTRA requires substantial adaptation to the target domain, whereas Flock’s structural encoding already captures relevant patterns from pretraining.

5.5.2 Classification versus Regression

Classification and regression tasks exhibit contrasting behavior that warrants examination. Classification performance remains at chance level ($\text{AUROC} \approx 0.5$) with oscillating training dynamics, while regression shows consistent improvement with MAE decreasing from 4.97 to 4.58.

This difference may stem from how each task uses candidate entities. Classification involves only two candidates (positive and negative outcomes), whereas regression involves multiple position candidates ranging from 1 to 20. The larger candidate set in regression provides more discriminative signal during training, enabling the model to learn meaningful rankings.

5.5.3 Train-Test Performance Gap

Table 7 reveals distinct failure patterns between the two architectures on classification tasks.

Table 7: Train-test performance comparison for classification tasks.

Task	Model	Train AUROC	Test AUROC
DNF	POSTRA	0.809	0.511
	Flock-time	0.507	0.498
Top3	POSTRA	0.974	0.539
	Flock-time	0.514	0.487

POSTRA achieves high training AUROC (0.809–0.974) but near-random test AUROC (0.511–0.539), indicating severe overfitting to training-specific patterns that do not generalize. Flock-time remains at chance level on both training and testing (approximately 0.5), indicating complete inability to extract discriminative signal from the graph structure. This contrast suggests that POSTRA’s entity-aware message passing can memorize training patterns, while Flock’s anonymized random walks cannot exploit such signals.

6 Conclusion

This work presents the first systematic evaluation of Knowledge Graph Foundation Models for relational database learning, comparing random-walk (Flock) and message-passing (ULTRA/POSTRA) paradigms across link prediction, classification, and regression tasks on RelBench F1.

Our experiments demonstrate that KGFM’s transfer effectively to relational domains for link prediction, with Flock achieving 0.894 MRR and outperforming ULTRA by 17.6% in zero-shot settings. For regression, both paradigms successfully predict continuous values when formulated as link prediction over discretized ranges. These results suggest that KGFM’s can reduce the need for task-specific model development on relational databases. For classification task, While POSTRA demonstrates a stronger capacity to capture training patterns through entity-aware message passing, it suffers from severe overfitting and fails to generalize to test data. In contrast, Flock’s anonymized random-walk approach appears unable to extract discriminative signals for binary outcomes, resulting in performance that remains near random chance across both training and testing phases.

Future work should address three directions: (1) correcting the experimental framework with proper train/test separation and expanded candidate sets, (2) validating findings across diverse RelBench datasets, and (3) exploring hybrid

architectures combining KGFM encoders with task-specific heads. As relational databases underpin much of the world’s structured data, developing robust foundation model approaches for this domain remains an important opportunity for the graph learning community.

References

- [1] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. *arXiv preprint arXiv:2310.04562*, 2023.
- [2] Jinwoo Kim, Xingyue Huang, Krzysztof Olejniczak, Kyungbin Min, Michael Bronstein, Seunghoon Hong, and İsmail İlkan Ceylan. Flock: A knowledge graph foundation model via learning on random walks. *arXiv preprint arXiv:2510.01510*, 2025.
- [3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural bellman-ford networks: A general framework for link prediction. *arXiv preprint arXiv:2106.06935*, 2021.
- [5] Jiaxin Pan, Mojtaba Nayyeri, Osama Mohammed, Daniel Hernandez, Rongchuan Zhang, Cheng Cheng, and Steffen Staab. Towards foundation model on temporal knowledge graph reasoning. *arXiv preprint arXiv:2506.06367*, 2025. Introduces POSTRA.
- [6] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, and Jure Leskovec. Relational deep learning: Graph representation learning on relational databases. *arXiv preprint arXiv:2312.04615*, 2023.
- [7] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC)*, 2018.
- [8] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Matthias Fey, Jan E. Lenssen, and Jure Leskovec. Relbench: A benchmark for deep learning on relational databases. *arXiv preprint arXiv:2407.20060*, 2024.
- [9] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [10] Jeff Heaton. An empirical analysis of feature engineering for predictive modeling. In *IEEE SoutheastCon*, 2016.
- [11] Komal K. Teru, Etienne Denis, and William L. Hamilton. Inductive relation prediction by subgraph reasoning. In *International Conference on Machine Learning (ICML)*, 2020.